

AIAC ASSIGNMENT – 8.1

NAME : E.ANJANI

HNO:2303A52419

BATCH:32

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

#CODE

```
Lab_ass_8.1 > Task1.py > ...
1 def is_strong_password(password):
2     if len(password) < 8:
3         return False
4     if " " in password:
5         return False
6     has_upper = any(c.isupper() for c in password)
7     has_lower = any(c.islower() for c in password)
8     has_digit = any(c.isdigit() for c in password)
9     has_special = any(not c.isalnum() for c in password)
10    return has_upper and has_lower and has_digit and has_special
11    assert is_strong_password("Abcd@123") == True
12    assert is_strong_password("abcd123") == False
13    assert is_strong_password("ABCD@1234") == True
```

#OUTPUT

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task1.py
Traceback (most recent call last):
  File "d:\courses\AIAC\Lab_ass_8.1\Task1.py", line 13, in <module>
    assert is_strong_password("ABCD@1234") == True
           ~~~~~^~~~~~
AssertionError
PS D:\courses\AIAC>
```

#MODIFIED CODE AND OUTPUT

```
Lab_ass_8.1 > Task1.py > ...
1 def is_strong_password(password):
2     if len(password) < 8:
3         return False
4     if " " in password:
5         return False
6     has_upper = any(c.isupper() for c in password)
7     has_lower = any(c.islower() for c in password)
8     has_digit = any(c.isdigit() for c in password)
9     has_special = any(not c.isalnum() for c in password)
10    return has_upper and has_lower and has_digit and has_special
11    assert is_strong_password("Abcd@123") == True
12    assert is_strong_password("abcd123") == False
13    assert is_strong_password("ABCD@1234") == False
```

```
● PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task1.py
All test cases passed!
○ PS D:\courses\AIAC> □
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

#CODE

```
Lab_ass_8.1 > Task2.py > ...
1 def classify_numbers(n):
2     if n is None or not isinstance(n, (int, float)):
3         return "Invalid input"
4     for i in range(1):
5         if n > 0:
6             return "Positive"
7         elif n < 0:
8             return "Negative"
9         else:
10            return "Zero"
11 assert classify_numbers(10) == "Positive"
12 assert classify_numbers(-5) == "Negative"
13 assert classify_numbers(0) == "Zero"
14 assert classify_numbers(None) == "Invalid input"
15 assert classify_numbers("abc") == "Invalid input"
16 print("All test cases passed!")
```

#OUTPUT

```
● PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task2.py
All test cases passed!
□
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

#CODE

```
Lab_ass_8.1 > Task3.py > ...
1 def is_anagram(str1, str2):
2     # Remove spaces and convert to lowercase for comparison
3     str1 = str1.replace(" ", "").lower()
4     str2 = str2.replace(" ", "").lower()
5
6     # If lengths are different, they can't be anagrams
7     if len(str1) != len(str2):
8         return False
9
10    # Count character frequencies in both strings
11    char_count = {}
12
13    # Count characters in first string
14    for char in str1:
15        char_count[char] = char_count.get(char, 0) + 1
16
17    # Subtract character counts based on second string
18    for char in str2:
19        if char not in char_count:
20            return False
21        char_count[char] -= 1
22        if char_count[char] == 0:
23            del char_count[char]
24
25    # If all counts are zero, strings are anagrams
26    return len(char_count) == 0
27 assert is_anagram("listen", "silent") == True
28 assert is_anagram("hello", "world") == False
29 assert is_anagram("Dormitory", "Dirty Room") == True
30 print("All test cases passed!")
```

#OUTPUT

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task3.py
● All test cases passed!
○ PS D:\courses\AIAC>
```

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

#CODE

```
Lab_ass_8.1 > Task4.py > ...
1 class Inventory :
2     def __init__(self):
3         self.items = {}
4     def add_item(self, name, quantity):
5         if name in self.items:
6             self.items[name] += quantity
7         else:
8             self.items[name] = quantity
9     def remove_item(self, name, quantity):
10        if name in self.items and self.items[name] >= quantity:
11            self.items[name] -= quantity
12            if self.items[name] == 0:
13                del self.items[name]
14        else:
15            raise ValueError("Not enough items to remove")
16    def get_stock(self, name):
17        return self.items.get(name, 0)
18 inv= Inventory()
19 inv.add_item("Pen", 10)
20 assert inv.get_stock("Pen") == 10
21 inv.remove_item("Pen", 5)
22 assert inv.get_stock("Pen") == 5
23 inv.add_item("Notebook", 3)
24 assert inv.get_stock("Notebook") == 3
25 print("All test cases passed!")
```

#OUTPUT

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task4.py				
● All test cases passed!				
○ PS D:\courses\AIAC>				

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

#CODE

```
demo.py • Task1.py Task2.py Task3.py Task4.py Task5.py
Lab_ass_8.1 > Task5.py > ...
1 def validate_and_format_date(date_str):
2     from datetime import datetime
3     try:
4         date_obj = datetime.strptime(date_str, '%m/%d/%Y')
5         return date_obj.strftime('%Y-%m-%d')
6     except ValueError:
7         return "Invalid date format. Please use MM/DD/YYYY."
8 assert validate_and_format_date("10/15/2023") == "2023-10-15"
9 assert validate_and_format_date("02/30/2023") == "Invalid date"
10 assert validate_and_format_date("01/01/2024") == "2024-01-01"
11 print("All test cases passed!")
```

#OUTPUT

```
PS D:\courses\AIAC> & C:/Python314/python.exe d:/courses/AIAC/Lab_ass_8.1/Task5.py
assert validate_and_format_date("02/30/2023") == "Invalid date "
~~~~~
AssertionError
PS D:\courses\AIAC> 
```